

Project Report: Secure Local Encrypted Password Vault Simulator

Course: Learn Depth Machine Learning Internship | **Author:** Intern Submission | **Date:** August 2026

1. Executive Summary & Problem Understanding

In modern computing environments, credential security remains one of the most critical vulnerabilities. Users routinely manage dozens of credentials, leading to password reuse or unencrypted plaintext storage in documents and spreadsheets. This project implements a secure, local, encrypted password vault simulator in Python demonstrating fundamental applied cybersecurity principles: **Authentication**, **Confidentiality**, **Integrity**, **Access Control**, and **Secure Credential Persistence**.

Core Security Requirements:

- **Confidentiality:** Stored records must be completely unreadable to unauthorized local users and malware.
- **Integrity:** Any bit-flip or tampering in the encrypted container must be immediately detected.
- **Authentication:** Master Password derivation must be memory-hard to prevent GPU/ASIC brute-forcing.
- **Access Control:** Decrypted data is strictly unavailable in memory while the vault is locked.

2. Proposed Approach & Cryptographic Architecture

The system implements envelope authenticated encryption using OWASP-recommended cryptographic standards:

- **Key Derivation (Argon2id):** Derives a 256-bit encryption key from the Master Password using 16-byte random salt, 64 MB memory cost, 2 time iterations, and 2 lanes. Fallback support for Scrypt is provided.
- **Authenticated Encryption (AES-256-GCM):** Encrypts the entire payload dictionary with a unique 12-byte nonce per write and a 128-bit authentication tag for tamper detection. Associated Data (AEAD) binds container metadata.
- **Defensive Memory Management:** In-memory credential records and encryption key buffers are zeroed and overwritten when locking or exiting.

3. Technical Implementation & Modular Structure

The project is architected into focused modular packages adhering to the Single Responsibility Principle:

Module	Responsibilities
<code>crypto/</code>	Implements Argon2id/Scrypt key derivation and AES-256-GCM AEAD encryption/decryption primitives.
<code>vault/</code>	Manages models (Credential, VaultPayload), atomic JSON storage, and CRUD coordinator.
<code>auth/</code>	Handles master password authentication, session state, and rate-limiting lockout defense.
<code>security/</code>	Provides password entropy evaluator, CSPRNG password generator, and secret-scrubbed audit logging.
<code>cli/</code>	User-friendly interactive terminal menu and direct CLI command interpreter.
<code>tests/</code>	Comprehensive pytest suite comprising 34 automated unit, integration, and security tests.

4. Important Technical Decisions

- **Argon2id vs PBKDF2:** Argon2id was selected for its memory hardness, providing mathematically superior resistance against parallelized GPU cracking hardware compared to CPU-only algorithms.
- **AES-256-GCM vs AES-CBC:** AES-GCM provides authenticated encryption with associated data (AEAD) in a single cryptographic primitive, eliminating separate HMAC key management and padding oracle attacks.
- **Single Encrypted Payload Envelope:** Storing all records inside a single encrypted container ensures that even account counts, service names, and usernames remain confidential on disk.
- **Secret-Scrubbed Audit Logging:** The audit logger automatically sanitizes log entries, replacing password and key keywords with [REDACTED] to prevent accidental leakage in diagnostic files.

5. Testing Performed & Verification

The implementation was thoroughly validated across **34 automated pytest test cases** with a 100% pass rate:

- **Cryptographic Tests (8 tests):** KDF determinism, salt randomness, AES-GCM encrypt/decrypt, bit-flip tampering rejection.
- **Authentication Tests (6 tests):** Password mismatch detection, short password rejection, lockout after 5 failed attempts.
- **Vault CRUD Tests (10 tests):** Add, Get (masked), Reveal, Update, Delete, List, Search, Duplicate rejection, Key rotation.
- **Storage Tests (5 tests):** Atomic save, envelope loading, corrupted file detection, backup, and restore.
- **Security Tests (4 tests): Plaintext Disk Inspection** asserting zero plaintext secrets exist in vault.enc.
- **End-to-End Tests (1 test):** Full execution sequence verifying all 12 project specification requirements.

6. Challenges Encountered & Solutions

1. *Balancing KDF security and latency:* Tuned Argon2id memory cost to 64 MB and iterations to 2, ensuring strong GPU defense while keeping derivation under 100 ms.
2. *Preventing accidental plaintext leakage:* Enforced default masking ('*****') across all views, requiring explicit user confirmation to reveal plaintext passwords.
3. *CLI Usability:* Created a smart resolver allowing users to select accounts by either service name or numerical list index.

7. Future Scope

Future iterations will include FIDO2/YubiKey hardware key support, a modern desktop graphical user interface (PyQt/Tauri), and breach detection via Have I Been Pwned API.